

Dealer Gamma Exposure (GEX) & Gamma-Flip Engine

Complete technical reference: architecture, quantitative methodology, every module and function, production hardening history, and interview preparation.

Component	File	LOC	Responsibility
Quant + CLI core	gex.py	1,489	BSM gamma, GEX aggregation, flip search, walls, Schwab data layer, rendering, charting, CLI
Test suite	test_gex.py	447	25 pytest cases: closed-form math validation, parser/filter behaviour, auth plumbing
HTTP API	server.py	373	FastAPI JSON endpoints, caching, client lifecycle, error classification
Mobile app	static/index.html	262	Installable iOS PWA: ticker/expiry controls, level cards, canvas chart
OAuth setup	scripts/schwab_setup.py	145	One-time browser login, token validation, live verification pull
Push notifier	scripts/daily_gex.sh	110	Multi-cadence runner, message assembly, ntfy/Pushover/Telegram delivery
Schedulers	scripts/*.plist	4 files	launchd agents: daily, weekly, monthly pushes + always-on API server

Stack: Python 3.9+ core (3.10+ for live data) · NumPy · SciPy · Matplotlib · FastAPI · Uvicorn · schwab-py (OAuth 2.0) · pytest · launchd · HTML/CSS/JS PWA

Data source: Charles Schwab Trader API (Market Data). **Instruments:** SPY, QQQ, and any optionable underlying with listed open interest.

1. Executive Summary

This project estimates **dealer gamma exposure** (GEX) from live options-chain data and derives the price levels where market-maker hedging flow changes character. Those levels form a daily trading bias: where volatility is likely to be suppressed, where it is likely to be amplified, and which strikes act as magnets or accelerants.

The 30-second pitch

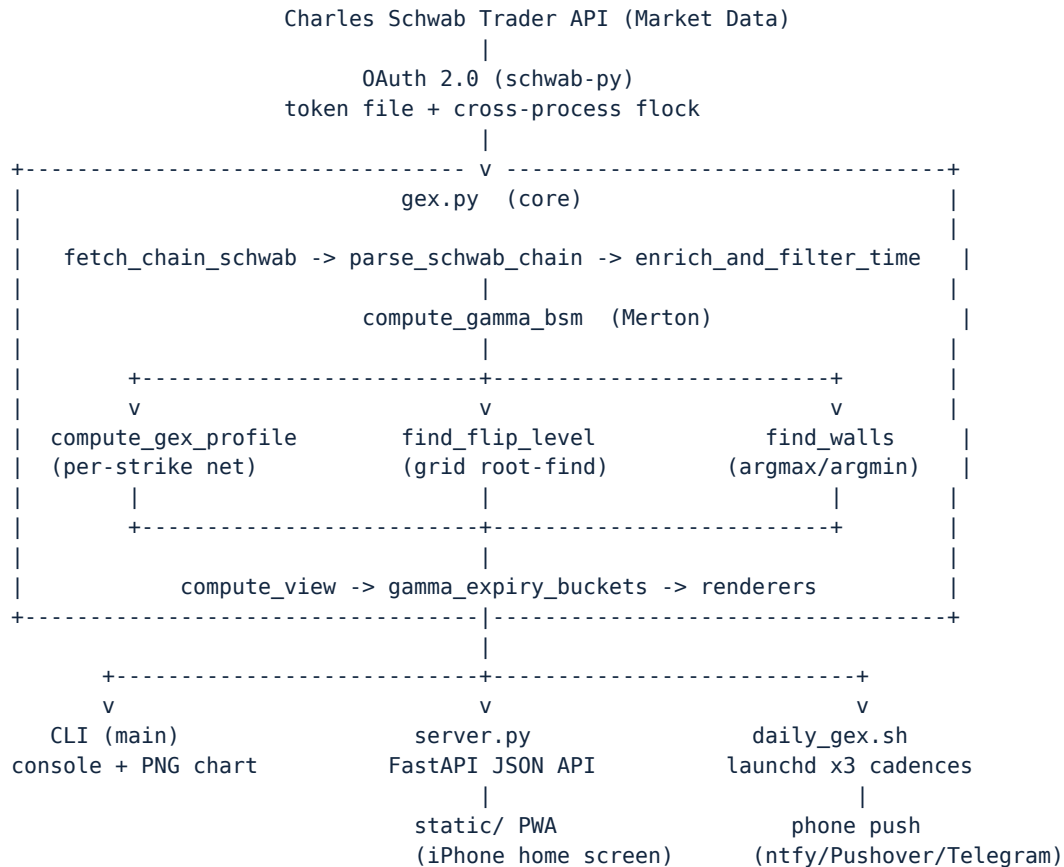
Options market makers hold inventory whose delta changes as spot moves. To stay hedged they must trade the underlying continuously, and the *direction* of that forced trading depends on the sign of their aggregate gamma. When dealers are **long gamma** they sell rallies and buy dips, damping volatility. When dealers are **short gamma** they buy rallies and sell dips, amplifying it. The system computes where that sign flips, and turns it into an actionable pre-market briefing delivered to a phone.

What makes this implementation different from a retail GEX chart

- **Gamma is recomputed from scratch** using Black-Scholes-Merton on per-strike implied volatility. Vendor greeks are never trusted, and the volatility smile is never flattened to a single at-the-money vol.
- **The flip is found by root-finding, not lookup.** Total GEX is repriced across a grid of hypothetical spot prices, recomputing both gamma and the S^2 dollar term at every node.
- **The core assumption is stress-tested every run.** The dealer sign convention is the model's biggest weakness, so the tool measures its own sensitivity to it and prints a LOW CONFIDENCE warning when the flip level is not robust.
- **Gamma is decomposed by hedging urgency.** A headline all-expiry number silently mixes 0DTE gamma that evaporates at 16:00 with six-month open interest that will never be rebalanced today.
- **It runs unattended in production.** Scheduled pushes, an HTTP API, and a phone app, hardened against real failures observed live: OAuth token-rotation races, trickling network responses, vendor data gaps.

2. Architecture and Data Flow

A single computational core (gex.py) is consumed by three independent delivery paths. No math is duplicated: the API, the scheduled notifier, and the CLI all call the same audited functions.



Design principle: one source of truth

Every delivery path is a thin adapter. server.py imports gex and calls the same functions the CLI does; the notifier shells out to the CLI. A change to the gamma formula propagates everywhere at once, and the test suite covers all three paths by covering the core.

3. The Quantitative Core

3.1 Step one: gamma per contract (Merton closed form)

Gamma is the second derivative of option value with respect to spot: the rate at which delta changes. It is identical for calls and puts at the same strike and expiry.

$$\text{gamma} = \exp(-q \cdot T) \cdot \phi(d_1) / (S \cdot \sigma \cdot \sqrt{T})$$

$$d_1 = [\ln(S/K) + (r - q + \sigma^2 / 2) \cdot T] / (\sigma \cdot \sqrt{T})$$

$$\phi(x) = \text{standard normal PDF} = \exp(-x^2 / 2) / \sqrt{2 \cdot \pi}$$

Symbol	Meaning	Source in this system
S	Spot price of the underlying	underlyingPrice from the chain snapshot (live)
K	Strike price	Per contract, from the chain
T	Time to expiry in years	Calendar ACT/365 to 16:00 ET, floored at 300 seconds
σ (sigma)	Implied volatility	Per-strike vendor IV, converted percent → decimal
r	Risk-free rate	--rate flag, default 0.043, always printed
q	Dividend yield	Per-ticker map (SPY 1.2%, QQQ 0.6%), overridable

Why the $\exp(-q \cdot T)$ factor matters. The dividend-adjusted (Merton) form carries an explicit discount factor that the plain Black-Scholes gamma omits. This was a genuine bug found during review: q appeared inside d_1 but not as the leading factor, which overstated gamma by roughly $q \cdot T$ whenever a dividend yield was supplied. Silent at $q=0$, wrong the moment a real yield was passed.

Implementation notes

- **Fully vectorized.** S, K, T and σ broadcast as NumPy arrays, so the entire chain is priced in one operation rather than a Python loop over thousands of contracts.
- **Degenerate inputs return 0, never NaN.** A boolean validity mask ($T > 0$, $\sigma > 0$, $S > 0$, $K > 0$) guarantees a sparse or malformed chain cannot poison aggregate levels with NaN or infinity.
- **Custom normal density.** `scipy.stats.norm.pdf` carries heavy per-call overhead. Because the flip search evaluates ϕ tens of millions of times per run, it was replaced with a direct NumPy expression, cutting runtime roughly $15\times$ (60s → 4s per ticker).

3.2 Step two: convert to dollar gamma

Raw gamma is unitless per share. It becomes economically meaningful when scaled to the dollar value of hedging flow the position forces:

dollar GEX per contract = $\text{gamma} * \text{open_interest} * \text{multiplier} * S^2 * 0.01$

```

gamma * S * 0.01    -> delta change per 1% move, per share
* S                -> dollar value of that delta change
* multiplier (100)  -> per contract
* open_interest     -> across all outstanding contracts

```

Unit convention warning (a real category error). This is the **per-1%** convention. The **per-point** convention is $\text{gamma} \times \text{OI} \times \text{multiplier} \times S$, which equals the per-1% figure divided by $(0.01 \times S)$. SqueezeMetrics publishes per-point; most retail charts publish per-1%. Comparing the two raw numbers is meaningless. The tool prints which convention it uses on every run.

3.3 Step three: apply the dealer sign convention

This is the model's single largest assumption and it is **not a fact**. The standard convention assumes customers net-buy puts as hedges and net-sell or buy calls such that dealers end up:

net GEX = $\text{SUM}(\text{call GEX}) - \text{SUM}(\text{put GEX})$ # dealers long calls, short puts

The signs are exposed as configuration (`DealerConvention.call_sign / put_sign`), flippable from the CLI. Ground truth would require dealer-direction data such as CBOE open-close volume, which the Schwab feed does not provide. Rather than pretend, the system quantifies its exposure to the

assumption (see 3.6).

3.4 Step four: aggregate per strike

Contracts across many expirations collapse onto a single sorted strike axis using a scatter-add, in one vectorized pass:

```
strikes = np.unique(K)           # sorted unique strike axis
idx      = np.searchsorted(strikes, K) # map each contract to its strike bucket
np.add.at(net, idx, signed)      # unbuffered scatter-add (handles duplicates)
```

Separate call, put, and net vectors are maintained so the profile chart can distinguish long-gamma (green) from short-gamma (red) strikes.

3.5 Step five: locate the gamma flip by root-finding

The flip, or zero-gamma level, is the spot price at which aggregate dealer gamma changes sign. It is **not** simply where the current profile crosses zero. It is found by asking: *if spot were somewhere else, what would total dealer gamma be?*

```
for each hypothetical spot S' in linspace(spot*0.90, spot*1.10, 1000):
    recompute gamma(S') for EVERY contract      # K, T, sigma, OI held fixed
    curve[i] = sum( sign * gamma(S') * OI * mult * S'^2 * 0.01 )
```

detect sign changes between adjacent nodes, then interpolate the exact crossing:

$$x = x_0 - y_0 * (x_1 - x_0) / (y_1 - y_0)$$

- **Both S-dependent terms are recomputed.** Gamma itself and the S^2 dollar-conversion factor both move with the hypothetical spot; holding either fixed produces a wrong level.
- **Multiple crossings are detected, not assumed away.** The GEX curve can legitimately cross zero more than once. All crossings are collected; the one nearest current spot is reported and the rest are listed.
- **Linear interpolation, not nearest-node.** The crossing is resolved between grid points rather than snapping to the grid, so precision does not depend on grid resolution alone.
- **±10% window.** Widened from ±5% during review because a deeply short-gamma session can push the flip beyond a narrow window, which would silently report 'no flip found'.

Sticky-strike caveat. Holding each contract's IV fixed while spot moves is the *sticky-strike* assumption. Reality sits somewhere between sticky-strike and sticky-delta, where the smile follows moneyness. The flip is therefore an estimate under a stated volatility-dynamics assumption, not a model-free level. This is documented in code and printed at runtime.

3.6 Uncertainty quantification: the sensitivity band

Because the dealer sign convention is an assumption, the system probes it two ways on every run:

Probe	Method	Interpretation
Literal sign flip	Recompute the flip with put_sign inverted (puts → long)	Usually makes <i>all</i> gamma positive, so the flip vanishes entirely. Reported as a structural caveat: a flip exists <i>only</i> because we assume dealers are short puts.
Graded magnitude band	Recompute the flip at ±50% of the short-put magnitude	Yields an actual distance: how far does the level move under a plausible mis-specification?

If the flip moves more than **1% of spot** across that band, a **LOW CONFIDENCE** warning is printed and surfaced as a badge in the mobile app. The philosophy: a model that cannot express doubt about its own central assumption is more dangerous than one that can.

3.7 Walls and regime

Output	Definition	Trading meaning
Call wall	Strike with the largest positive net GEX	Resistance / pin. Dealer hedging suppresses movement through it; rallies often stall.
Put wall	Strike with the largest negative net GEX	Support that becomes a downside <i>accelerant</i> if breached, because the same concentration flips from cushion to fuel.
Regime	spot > flip → LONG gamma; spot < flip → SHORT gamma	Long: vol-damping, mean-reverting, fade extremes. Short: vol-amplifying, trend-prone, breaks run.

3.8 Hedging-urgency decomposition

The governing discipline: **match the expiry set to the holding period**. An all-expiry total silently mixes gamma that must be reheded within hours with gamma that will not be touched for months. Every run prints a first-match bucket table:

Bucket	Why it matters
0DTE (expires today)	Enormous intraday gamma that evaporates completely at 16:00 ET . Also the bucket where prior-close OI is least trustworthy.
≤ 1 week	Fast money; genuinely reheded on the horizon most intraday traders care about.
≤ monthly OpEx (3rd Friday)	Holds the bulk of index open interest. Charm and vanna dominate into roll-off (not modelled here).
Beyond OpEx	Slow money. Will not be rebalanced today; inflates headline numbers without driving today's flow.

Monthly OpEx dates are derived from first principles (third Friday via weekday arithmetic, with correct year rollover) rather than hardcoded.

4. Module Reference: gex.py

1,489 lines, organized into labelled sections. Private helpers are prefixed with an underscore.

4.1 Data model

Object	Purpose
Contract	Dataclass: strike, expiry, type (call/put), open interest, IV, and T (filled during enrichment). One row of the filtered chain.
DealerConvention	Dataclass: call_sign, put_sign, and a human-readable label. Makes the central assumption a first-class, swappable object rather than a magic number.
Config	Dataclass: ticker, multiplier, rate, div_yield (+ its provenance string), price_range, grid steps, and both conventions. Threaded through every computation so nothing reads globals.

4.2 Pricing and aggregation

Function	Contract
<code>_norm_pdf(x)</code>	Vectorized standard normal density. Replaces scipy for a ~15× speedup in the hot path.
<code>compute_gamma_bsm(S,K,T,σ,r,q)</code>	Merton gamma, fully broadcastable. Returns 0 for degenerate inputs. The single most tested function in the codebase.
<code>_to_arrays(contracts, conv)</code>	Packs a contract list into parallel NumPy arrays (K, T, iv, oi, sign, iscall). The sign vector is where the dealer convention is materialized.
<code>_signed_dollar_gex(...)</code>	Per-contract signed dollar GEX at a given spot: $\text{gamma} \times \text{OI} \times \text{mult} \times S^2 \times 0.01 \times \text{sign}$.
<code>compute_gex_profile(...)</code>	Scatter-add aggregation onto the sorted strike axis. Returns strikes, net, call_gex, put_gex, and the scalar total.
<code>gross_dollar_gamma(...)</code>	Sum of dollar gamma , sign-agnostic. Used for concentration and bucket-share calculations.

4.3 Flip and walls

Function	Contract
<code>find_flip_level(...)</code>	Grid root-finder. Returns the nearest crossing, <i>all</i> crossings, the full (grid, curve) arrays for plotting, and total GEX at spot.
<code>_flip_with_put_sign(...)</code>	Helper for the graded sensitivity band: recomputes the flip at a scaled put sign.
<code>find_walls(profile)</code>	argmax / argmin over the net profile. Returns both wall strikes and their GEX magnitudes.
<code>compute_view(...)</code>	Orchestrator for one slice of the chain: profile + walls + standard flip + flipped flip + sensitivity band + gross gamma.

4.4 Time handling

Function	Contract
<code>_et_tz() / now_et()</code>	America/New_York via zoneinfo, so daylight-saving transitions are handled correctly rather than by a fixed UTC offset.
<code>seconds_to_expiry(exp, now)</code>	Seconds until 16:00 ET on the expiration date (PM settlement at the cash close).
<code>enrich_and_filter_time(...)</code>	Fills T in years, drops already-settled contracts, and floors T at 300 seconds. Returns counts of both so the caller can report them.
<code>third_friday / next_monthly_opex</code>	Monthly OpEx calendar arithmetic, including year rollover, driving the expiry buckets.
<code>prior_trading_session(today)</code>	Most recent weekday, used to label the open-interest reference date (holidays not modelled).

Why the T floor exists. Gamma carries $S \cdot \sigma \cdot \sqrt{T}$ in the denominator, so as $T \rightarrow 0$ at-the-money gamma diverges toward infinity. Without a floor, a 0DTE chain minutes before the close would produce meaningless astronomical levels. Clamped contracts are counted and the warning is printed.

4.5 Data layer (Schwab)

Function	Contract
<code>to_schwab_symbol(t)</code>	Maps friendly tickers to Schwab symbology (SPX → \$SPX); ETFs pass through unchanged.
<code>get_schwab_client(...)</code>	Builds a schwab-py client from the cached token. Lazy import (schwab-py needs Python 3.10+, the core runs on 3.9). Sets a 30s HTTP timeout and stamps the token path + mtime onto the client.
<code>token_lock(path)</code>	Cross-process exclusive flock on a sidecar file, serializing API calls so concurrent refreshes cannot rotate each other's tokens out of existence. Degrades to unlocked on timeout (a race beats a guaranteed outage).
<code>schwab_client_stale(client)</code>	True if the token file changed on disk since the client was built. Lets long-lived processes discard a superseded refresh token.
<code>fetch_chain_schwab(...)</code>	One un-paginated chain request with retry logic that classifies failures by recoverability .
<code>parse_schwab_chain(data)</code>	Traverses the nested expiry→strike→contract maps and applies every data-quality filter. Returns contracts, spot, timestamp, drop counts, and vendor status.
<code>fetch_spx_spy_ratio(...)</code>	Live SPX/SPY ratio, computable from either leg, replacing a hardcoded 10× approximation.

Data-quality filters applied in `parse_schwab_chain`

Filter	Rule	Rationale
Zero open interest	Drop $OI \leq 0$	GEX is OI-weighted; a strike with no OI contributes nothing.

Filter	Rule	Rationale
Missing / sentinel IV	Drop non-finite or ≤ 0 (catches Schwab's -999.0)	Garbage IV in, garbage gamma out.
Crossed quotes	Drop bid > ask	A crossed market is stale or locked data.
Deep-ITM bad quotes	Drop if >5% ITM and (no bid or relative spread >25%)	Deep-ITM IV is extracted from a sliver of extrinsic value and is noise-dominated.
OTM wings	Never quote-filtered	Zero-bid wings still carry real tail gamma. Filtering them would destroy wing gamma, recreating the flattened-smile bug from the other direction.

4.6 Rendering and output

Function	Contract
<code>print_assumptions(...)</code>	Prints every modelling choice at runtime: data source, IV handling, exercise style, time clock, quote filters, r, q + provenance, convention, unit warning.
<code>print_data_health(...)</code>	Spot, snapshot age, contract counts, per-reason drop counts, T-floor warnings, and the OI staleness caveat with its reference date.
<code>print_gamma_buckets(...)</code>	Renders <code>gamma_expiry_buckets()</code> as the hedging-urgency table.
<code>print_view_detail(...)</code>	Flip (with cross-quote), walls, and the full put-sign sensitivity analysis including the LOW CONFIDENCE verdict.
<code>render_summary(...)</code>	The plain-text bias block: spot, regime, flip, walls, total GEX, and a one-line interpretation.
<code>render_levels_compact(...)</code>	Terse block for phone notifications (--levels-only).
<code>cross_quote(...)</code>	Bidirectional SPX↔SPY level translation; returns None for unrelated tickers so no spurious cross-quotes appear.
<code>plot_profile(...)</code>	Matplotlib bar chart: green/red per-strike GEX, with spot, flip, and both walls marked. X-axis ticks snap to round multiples of 10, auto-coarsening (10→20→50→100) when a window would exceed ~25 ticks.

4.7 CLI surface

Flag	Effect
<code>--ticker</code>	Underlying (default SPY).
<code>--expiry</code>	0dte all YYYY-MM-DD. Default computes 0DTE <i>and</i> all expiries side by side.
<code>--all-days</code>	Expiration window for 'all' (default 45; wider windows risk a vendor 502).
<code>--rate / --div-yield</code>	Override r and q; both are echoed with their provenance.

Flag	Effect
--convention / --call-sign / --put-sign	Flip or hand-set the dealer sign assumption.
--price-range / --steps	Flip-search window and grid resolution.
--levels-only	Compact output for notifications (still saves the chart).
--x-tick / --no-plot / --out-prefix	Chart controls.
--demo	Deterministic synthetic chain; runs with no credentials and no network.

5. Delivery Layer

5.1 server.py — FastAPI JSON API

Endpoint	Returns
GET /api/health	Token age, days remaining, and the last real auth outcome (not just file age, which lies when a token family has been revoked).
GET /api/expirations?ticker=	Real listed expirations for the picker, with already-settled dates excluded so an empty-chain selection is impossible.
GET /api/gex?ticker=&expiry=&all_days=	Full payload: spot, per-view levels (flip, crossings, walls, regime, interpretation, low-confidence flag), the per-strike profile, expiry buckets, drop counts, and warnings.

Server-side engineering

- **Result caching** (60s TTL, keyed by ticker/expiry/window/rate/q) so pull-to-refresh cannot hammer the vendor.
- **Client lifecycle:** the cached Schwab client is discarded when the token file changes on disk, and again on any auth error, so the server heals itself after a re-login with no restart.
- **Bounded fetch:** a ThreadPoolExecutor deadline caps total wall-time per request, because a slow-trickling response can evade socket-level read timeouts entirely.
- **Error classification** maps exceptions to meaningful HTTP status plus a machine-readable code (schwab_token_expired, schwab_unreachable, bad_expiry, no_spot), which the app renders as actionable messages.

5.2 static/ — the iPhone PWA

A single self-contained HTML file with no build step and no external dependencies. Installs to the home screen via a web app manifest and renders as a standalone dark-mode app.

- **Controls:** SPY/QQQ chips plus free-text ticker; expiry dropdown populated live from the API; manual refresh and an optional 60-second auto-refresh.
- **Display:** regime banner (green long-gamma / red short-gamma) with LOW CONFIDENCE badge, six level cards, a hand-rolled canvas chart, and the expiry-bucket table.
- **Canvas chart** drawn directly with the 2D context: device-pixel-ratio scaling for retina, round-number gridlines with the same auto-coarsening logic as the matplotlib chart, and marker lines for spot, flip, and both walls.
- **Resilience:** client-side fetch deadline with AbortController, redraw on resize and visibility change (a canvas measured while hidden has zero width and renders blank).

5.3 Notification pipeline

scripts/daily_gex.sh <cadence> runs the CLI for each ticker, assembles a compact message, and delivers it. Three cadences widen the expiration window as the horizon lengthens:

Cadence	Schedule (local CT / ET)	Window	Rationale
daily	Mon-Fri 07:45 / 08:45	45 days	Near-term tactical levels, ~45 min before the open.
weekly	Monday 07:50 / 08:50	90 days	Structural view into the next quarter.
monthly	1st of month 07:55	150 days	Long-horizon reset; captures the next several OpEx cycles.

- **Three notifier backends** (ntfy, Pushover, Telegram), auto-selected by which credentials are present.
- **Failure is itself a notification.** Titles distinguish success, PARTIAL (one ticker failed), RUN FAILED, and SCHWAB TOKEN EXPIRED with the exact remediation command.
- **Proactive token warning:** from day 5.5 of the 7-day refresh-token lifetime, every push appends a warning so re-login happens on schedule rather than as a morning outage.
- **Scheduling via launchd** user agents; the always-on API server runs under a KeepAlive agent that restarts it on crash and at login.

6. Assumptions, Limitations, and Deliberate Deviations

Being able to articulate these precisely is the difference between having built a tool and understanding one. Each item below is documented in code and printed at runtime.

6.1 Standing assumptions

Assumption	Status and impact
Dealer sign convention	The biggest weakness. Long calls / short puts is an inference, not observed data. The flip's very <i>existence</i> depends on it. Mitigated by the sensitivity band and LOW CONFIDENCE flag, not solved.
Open interest is prior-session	OCC publishes end-of-day; the freshest OI available is yesterday's close. ODTE positions opened today are invisible. Printed with its reference date on every run.
Sticky-strike repricing	Per-strike IV held fixed across the flip grid. Reality lies between sticky-strike and sticky-delta.
European exercise	BSM gamma is used for American ETF options. The early-exercise premium concentrates in deep-ITM (low-gamma) strikes and is small for the short-dated flow that dominates GEX.
Charm and vanna ignored	Second-order flows dominate into monthly OpEx roll-off and are explicitly out of scope.

6.2 Deliberate deviations from the textbook pipeline

Two conventional prescriptions were **deliberately not followed**, with reasoning:

Calendar time, not trading time

A common prescription is to measure 0DTE T as $\text{minutes_to_1600} / (390 \times 252)$ trading minutes. This system uses calendar ACT/365 instead, because gamma depends on the product $\sigma^2 \cdot T$ and the input IVs are **vendor quotes annualized on a calendar clock**. Pairing a calendar-annualized sigma with a trading-time T breaks that pairing and would inflate 0DTE gamma severalfold. A trading-time clock is correct only when IV is re-derived from prices under the same clock. Internal consistency beats superficially more sophisticated.

Single underlying, not the full index complex

A complete S&P dealer book would aggregate SPX + SPXW + XSP + SPY + ES options normalized to common notional. That is impossible on this data source: Schwab returns **zero open interest for cash-index options** and carries no futures options. The tool therefore measures one listed underlying as a correlated proxy, states the limitation explicitly, and **refuses to sum raw GEX across underlyings**, which without notional normalization would be a meaningless number.

6.3 Hard data-source constraints discovered empirically

Constraint	Evidence and consequence
No index open interest	Verified live: every \$SPX strike returns OI = 0 while SPY returns full OI (max ~37k). Index GEX is impossible on Schwab . Default ticker was changed to SPY and the tool now emits a warning pointing at ETFs.
Chain size limits	A full-year SPX chain returns HTTP 502. Default window reduced to 45 days with a 502-specific hint.
7-day refresh token	Schwab refresh tokens expire weekly and require a browser re-login; there is no headless renewal path. Surfaced proactively rather than hidden.
No dealer-direction data	No open-close volume feed, so the sign convention cannot be empirically grounded.

7. Testing Strategy

25 pytest cases, no network access required. The philosophy: **financial code must be validated against independently derived truth**, not against its own output.

7.1 Mathematical validation against closed-form truth

Test	Reference value and derivation
ATM gamma, clean case	$S=K=100, T=1, \sigma=0.2, r=q=0$ gives $d_1=0.1$ analytically; expected gamma 0.0198476275, asserted to 1e-9.
Hull textbook example	$S=49, K=50, r=0.05, \sigma=0.20, T=20/52 \rightarrow$ gamma ≈ 0.066 as published in Hull's <i>Options, Futures, and Other Derivatives</i> . An external reference, not self-generated.
Dividend discount	Constructed so $d_1=0$ exactly ($r=0, q=0.02, \sigma^2/2$ cancels), isolating the $\exp(-q \cdot T)$ factor; asserted to 1e-12.
Degenerate inputs	$T \leq 0, \sigma \leq 0, S \leq 0$ must return exactly 0, never NaN or infinity.
Vectorization	Array broadcast returns the right shape and ATM gamma is the maximum across strikes.

7.2 The flip test: a hand-derived analytical crossing

The most valuable test in the suite. For one call at K_c and one put at K_p with equal OI and identical σ and T , the shared $1/(S \cdot \sigma \cdot \sqrt{T})$ and S^2 factors cancel at the zero, so the crossing occurs where $\phi(d_1^c) = \phi(d_1^p)$, i.e. $d_1^c = -d_1^p$. Solving gives a closed form:

$$S^* = \sqrt{K_c \cdot K_p} \cdot \exp(-c \cdot T), \quad \text{where } c = r - q + \sigma^2 / 2$$

$$K_c = 90, K_p = 110, \sigma = 0.2, T = 1, r = q = 0 \rightarrow c = 0.02$$

$$S^* = \sqrt{9900} \cdot \exp(-0.02) = 97.5288$$

The grid root-finder must land on 97.5288 within 0.05. This validates the interpolation, the sign-change detection, and the nearest-crossing selection against mathematics rather than a golden output file. Companion tests assert that no crossing exists when all gamma is positive (the flipped-sign degeneracy) and that the flip is invariant to multiplier and OI scaling.

7.3 Data layer and infrastructure tests

Area	What is asserted
Chain parsing	A hand-built payload in Schwab's exact nested shape: expiry read from the map key, IV percent→decimal, -999 sentinel dropped, OI=0 dropped, spot and ms→ns timestamp conversion.
Quote filters	Crossed quote dropped; deep-ITM no-bid and wide-spread dropped; deep-OTM zero-bid wing preserved (the subtle case that protects tail gamma).
Walls and conventions	Extreme-strike selection; total GEX sign inverts exactly under the flipped convention.
Cross-quoting	SPX→SPY divides, SPY→SPX multiplies, unrelated tickers return None.
Dividend resolution	Per-ticker map applies, explicit flag overrides, unknown ticker falls back to 0.

Area	What is asserted
OpEx calendar	Third Friday correct across months, on the day itself, and across a year boundary.
Retry classification	Transient timeouts retry and then succeed; OAuth errors and 4xx raise immediately without retrying.
Token lock	A separate process is genuinely blocked while the lock is held, and released afterwards; lock timeout degrades to unlocked rather than failing the run.
Client staleness	A rewritten token file is detected via mtime so a long-lived process discards a superseded token.

Runtime: the full suite completes in well under a second, so it runs on every change without friction.

8. Production Hardening: Bugs Found and Fixed

Every item below was discovered by running the system against live markets, not in review. This section is the strongest interview material in the project because each entry has a symptom, a root cause, and a verified fix.

8.1 Quantitative correctness

Defect	Root cause	Fix
Gamma overstated when a dividend yield was supplied	The Merton $\exp(-q \cdot T)$ leading factor was missing; q appeared only inside d_1	Added the factor; regression test constructed so $d_1=0$ isolates it exactly
SPX-equivalent levels off by ~25 points	SPX/SPY cross-quote used a hardcoded 10.0; the live ratio was 10.034	Live ratio fetched from either leg of the pair, with the source label printed
Flip occasionally reported as 'not found'	$\pm 5\%$ search window too narrow for deeply short-gamma sessions	Widened to $\pm 10\%$
Labels read 'SPX' on SPY runs	Ticker name hardcoded in the renderers	All labels derive from config; cross-quotes suppressed for unrelated tickers

8.2 Performance

Symptom: a two-ticker scheduled run took roughly two minutes. **Diagnosis:** the flip search evaluates the normal PDF across 1,000 grid nodes $\times$ thousands of contracts, on the order of tens of millions of calls per run, and `scipy.stats.norm.pdf` carries substantial per-call overhead. **Fix:** a direct NumPy density expression. **Result:** 60s $\rightarrow$ 4s per ticker ($\sim 15\times$), whole pipeline 8s, with identical numerical output verified by the existing tests.

8.3 Distributed-systems and reliability

Defect	Root cause	Fix
Fresh OAuth tokens dying in ~1 day instead of 7	A long-lived server process cached a Schwab client in memory indefinitely. After a re-login rewrote the token file, that process kept presenting the superseded refresh token ; Schwab treats reuse as compromise and revokes the entire token family, killing the brand-new token	mtime-based staleness detection so holders rebuild, plus a cross-process flock serializing refreshes
A single request hung for 868 seconds	httpx read-timeouts fire only on socket silence; a response trickling bytes slowly never triggers one	Hard wall-clock deadline via ThreadPoolExecutor, independent of socket activity
Scheduled runs died on transient network hiccups	Any exception aborted the ticker	Retry with linear backoff for transient faults, immediate raise for auth and 4xx where retrying cannot help
Scheduled jobs silently never executed	macOS TCC: launchd's bash had no Full Disk Access to the project directory	Diagnosed via launchd stderr; documented the permission grant and verified by kickstart
A login 'succeeded' but wrote an unusable token	A bad authorization code yields a degenerate short-lived token with no refresh token	Post-login validation rejects tokens lacking a refresh token, failing loudly at setup instead of at 07:45 the next morning
Health endpoint reported a healthy token while every call failed	Health was computed from file age alone	Health now reflects the last real API outcome

8.4 Data-source discovery

The most consequential finding of the project. The tool was originally written for SPX. Live testing showed **every \$SPX strike returning zero open interest** while SPY returned full OI on the same request pattern. Since GEX is OI-weighted, index GEX is simply not computable from this vendor. The response was to change the default underlying to SPY, add an explicit diagnostic pointing users to ETFs, and document the limitation prominently rather than silently emitting empty results. A related discovery: requesting a full year of expirations returns HTTP 502, which drove the 45-day default window.

9. Anticipated Questions

Prepared answers to the questions this project invites.

Q. Why compute gamma yourself instead of using the broker's greeks?

Three reasons. First, auditability: I cannot verify a vendor's model, its rate assumption, or its dividend treatment, and every one of those changes the number. Second, consistency: vendor greeks across strikes may be computed at different times or with different conventions. Third, control: recomputing lets me change the risk-free rate, dividend yield, or exercise-style assumption and see the effect. The vendor IV is still an input, so this does not eliminate vendor dependence, it isolates it to one clearly labelled place.

Q. What is the biggest source of error in your output?

The dealer sign convention, without question. Assuming dealers are long calls and short puts is an inference about positioning, not measured data, and the gamma flip's very existence depends on it. If you flip the put sign, all gamma becomes positive and the flip vanishes entirely. That is why the tool reports a sensitivity band across $\pm 50\%$ of the short-put magnitude and prints LOW CONFIDENCE when the level moves more than 1% of spot. The honest framing is that this produces directional regime context, not a precise tradeable level.

Q. Why SPY instead of SPX, when SPX is the real index-options market?

It was forced by the data source, and I verified it rather than assuming. Schwab returns zero open interest for cash-index options while SPY returns full OI on the identical request. Since GEX is OI-weighted, SPX GEX is not computable from this vendor at any price. SPY is the standard proxy, so the tool runs on SPY and cross-quotes levels to SPX terms using the live ratio. I document that this is a proxy for one underlying, not the full SPX complex, and I deliberately refuse to sum GEX across underlyings without notional normalization.

Q. How do you know your flip level is actually correct?

I test it against a closed-form solution I derived by hand. For a single call and a single put with equal OI and matching vol and tenor, the common factors cancel at the zero, so the crossing must occur where the two normal densities are equal, giving $S^* = \sqrt{K_c \cdot K_p} \cdot \exp(-cT)$. The grid search has to land on 97.5288 for my chosen parameters. That validates the interpolation and crossing detection against mathematics rather than against a previously recorded output.

Q. Why calendar time rather than trading time for ODTE?

Because I ingest vendor IV rather than deriving it. Gamma depends on the product sigma-squared times T, and the vendor's IV is annualized on a calendar clock. If I pair that sigma with a trading-time T of 390 times 252, I break the pairing and inflate ODTE gamma severalfold. Trading time is the better convention only if you re-derive IV from prices under the same clock. Given my inputs, calendar time is the internally consistent choice, and I documented that reasoning in the code.

Q. What does this tool NOT tell you?

Several things, all stated in the output. It does not capture positions opened today, because open interest is a prior-session figure, which matters most exactly where gamma is largest, in ODTE. It does not model charm or vanna, which dominate into monthly OpEx roll-off. It does not measure the full index complex. And it is a regime classifier, not an entry trigger.

Q. How would you improve it with a better data budget?

Three upgrades in priority order. First, CBOE open-close volume data to ground the dealer sign convention empirically instead of assuming it, which attacks the single largest error source. Second, real-time signed volume for intraday ODTE, where stale open interest is genuinely the wrong input. Third, a vendor with index and futures-options open interest so the full SPX complex could be aggregated on common notional.

Q. Talk me through the hardest bug you fixed.

OAuth tokens were dying in about a day when they should last seven, and re-logging in only bought another day. The clue was that the token file's modification time never changed after creation, so no successful refresh had ever been written, yet the refresh token was revoked. The cause was a long-running server process caching a client in memory. After I logged in fresh, that old process was still holding the previous refresh token and presenting it. Schwab treats a superseded refresh token as a compromise signal and revokes the whole family, so an old process was killing every new login. The fix was two parts: detect that the token file changed on disk and rebuild the client, and add a cross-process file lock so concurrent refreshes cannot rotate each other's tokens out of existence.

Q. How do you test code where the correct answer is not obvious?

I look for cases where I can derive truth independently. Closed-form solutions with parameters chosen so terms cancel, published textbook reference values like the Hull example, invariants that must hold such as the flip being independent of contract multiplier, and degenerate inputs with a known-required output. For infrastructure I test observable behaviour, for example spawning a real subprocess to prove the file lock actually excludes another process.

Q. Why did you make it a phone app rather than just a script?

Because the levels are only useful in a narrow window before the open, and a tool that requires me to sit at a terminal at 07:45 is a tool I will skip. The scheduled push makes the analysis arrive whether or not I remember, and the app exists for when I want to change the ticker or look at a specific expiration intraday. It also forced a cleaner architecture, since building an API on top of the core proved the computation was genuinely separable from the presentation.

Q. What would you do differently if you started over?

I would validate the data source against the actual requirement before building on it. I built for SPX and discovered only at live-testing time that the vendor has no index open interest, which invalidated the original target instrument. A ten-minute check of one field would have saved a migration. Second, I would have written the token-sharing model deliberately from the start rather than growing from one process to four and discovering the refresh-rotation race in production.

10. Quick Reference

Key constants and defaults

Constant	Value	Meaning
DEFAULT_TICKER	SPY	Index options have no OI on this vendor
DEFAULT_MULTIPLIER	100	Standard equity/ETF and index contract multiplier
DEFAULT_RATE	0.043	Risk-free rate; always printed, override with --rate

Constant	Value	Meaning
TICKER_DIV_YIELDS	SPY 0.012, QQQ 0.006	Auto-resolved dividend yields with printed provenance
DEFAULT_PRICE_RANGE	0.10	±10% flip-search window
DEFAULT_GRID_STEPS	1000	Flip-search grid resolution
DAY_COUNT / EXPIRY_HOUR_ET	365 / 16:00 ET	Calendar ACT/365 to the cash close
T_FLOOR_SECONDS	300	Bounds the 1/sqrt(T) singularity at the close
MATERIAL_FLIP_MOVE	0.01	LOW CONFIDENCE threshold: 1% of spot
ITM_DEPTH_FILTER	0.05	'Deep ITM' means more than 5% in the money
MAX_ITM_REL_SPREAD	0.25	Deep-ITM relative spread above this is treated as stale
FETCH_DEADLINE_S	90	Hard wall-clock cap on a chain fetch

Operating commands

```
# one-time / weekly (Schwab refresh tokens last 7 days)
python3 scripts/schwab_setup.py --manual

# daily levels
python3 gex.py --ticker QQQ                # 0DTE + all expiries
python3 gex.py --ticker SPY --expiry 0dte  # single view
python3 gex.py --demo                      # offline, no credentials

# services
python3 server.py                          # API + phone app on :8787
bash scripts/daily_gex.sh daily|weekly|monthly # push to phone
python3 -m pytest test_gex.py              # 25 tests
```

Glossary

Term	Definition
Gamma (Γ)	Rate of change of delta with respect to spot. The second derivative of option value.
Dealer gamma exposure	Aggregate dollar gamma held by market makers; determines the direction of their forced hedging flow.
Gamma flip / zero gamma	The spot level where aggregate dealer gamma changes sign, and with it the character of hedging flow.
Long gamma regime	Dealers sell rallies and buy dips. Volatility-damping, mean-reverting.
Short gamma regime	Dealers buy rallies and sell dips. Volatility-amplifying, trend-prone.
Call wall / put wall	Strikes with the largest positive / negative net GEX; act as pin-resistance and support-then-accelerant.

Term	Definition
ODTE	Options expiring the same session. Very large gamma that disappears entirely at 16:00 ET.
OpEx	Monthly options expiration, the third Friday, which holds the bulk of index open interest.
Sticky strike	Assumption that each strike's implied vol stays fixed as spot moves.
Charm / vanna	Second-order greeks: delta decay over time, and delta sensitivity to volatility.